H1N1 VACCINATION PREDICTION (major project-1)

IMPORTING

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

Loading data

```
df = pd.read_csv
```

EDA And Preprocessing

```
df.head()
```

```
# Assuming the uploaded file is named 'your_uploaded_file.csv'
file_name = next(iter(uploaded))
df = pd.read_csv(io.BytesIO(uploaded[file_name]))

display(df.head())
```

```
df.shape
```

```
sns.pairplot(df)
```

```
df.info()
```

```
# deleting not necessary features for prediction
df.drop(['unique_id','race','income_level','marital_status','housing_status','employment','qualification','census_msa'],axi
```

```
df.isnull().sum()
```

```
sns.lineplot(df['age_bracket'])
```

```
df.shape
```

Handling Missing Value

h1n1_worry

```
df['h1n1_worry'].unique()
```

Worry about the h1n1 flu(0,1,2,3)

0=Not worried at all,

1=Not very worried,

2=Somewhat worried,

3=Very worried

```
sns.boxplot(df['h1n1_worry'])
```

AS data is normally distributed and there is no outliers and its categorical data we will fillup the misssing value with mode of h1n1_worry

```
df['h1n1_worry']=df['h1n1_worry'].fillna(df['h1n1_worry'].mode()[0])
```

```
df['h1n1_worry'].value_counts()
```

h1n1_awareness

```
df['h1n1_awareness'].unique()
```

Signifies the amount of knowledge or understandingthe respondent has about h1n1 flu - (0,1,2) 0=No knowledge,

1=little knowledge

2=good knowledge

```
df['h1n1_awareness'].value_counts()
```

```
sns.boxplot(df['h1n1_awareness'])
```

As data is categorical we filling up with mode

```
df['h1n1_awareness']=df['h1n1_awareness'].fillna(df['h1n1_awareness'].mode()[0])
```

```
df['h1n1_awareness'].value_counts()
```

antiviral_medication

```
df['antiviral_medication'].unique()
```

Has the respondent taken antiviral vaccination - (0,1)

```
sns.displot(x=df['antiviral_medication'])
```

```
df[df['antiviral_medication']==1.0]
```

```
df['antiviral_medication'] = df['antiviral_medication'].fillna(df['antiviral_medication'].mode()[0])
```

```
df['antiviral_medication'].value_counts()
```

contact_avoidance

```
df['contact_avoidance'].unique()
```

```
sns.boxplot(df['contact_avoidance'])
```

```
df['contact_avoidance'] = df['contact_avoidance'].fillna(df['contact_avoidance'].mode()[0])
```

```
df['contact_avoidance'].value_counts()
```

```
df.isnull().sum()
```

```
df['bought_face_mask'].unique()
```

```
df.drop(['bought_face_mask','wash_hands_frequently','avoid_large_gatherings','reduced_outside_home_cont','avoid_touch_face
```

```
df.isnull().sum()
```

```
df['dr_recc_h1n1_vacc'].unique()
```

```
df['dr_recc_h1n1_vacc']= df['dr_recc_h1n1_vacc'].fillna(df['dr_recc_h1n1_vacc'].mode()[0])
```

```
df['dr_recc_h1n1_vacc'].value_counts()
```

```
df['dr_recc_seasonal_vacc'].unique()
```

```
df['dr_recc_seasonal_vacc']= df['dr_recc_seasonal_vacc'].fillna(df['dr_recc_seasonal_vacc'].mode()[0])
```

```
df['chronic_medic_condition'].unique()
```

```
df['chronic_medic_condition']= df['chronic_medic_condition'].fillna(df['chronic_medic_condition'].mode()[0])
```

```
df['is_health_worker'].unique()
```

```
df['is_health_worker']= df['is_health_worker'].fillna(df['is_health_worker'].mode()[0])
```

```
df['has_health_insur'].unique()
```

```
df['has_health_insur']= df['has_health_insur'].fillna(df['has_health_insur'].mode()[0])
```

```
df['is_h1n1_vacc_effective'].unique()
```

```
sns.boxplot(df['is_h1n1_vacc_effective'])
```

```
df['is_h1n1_vacc_effective'] = df['is_h1n1_vacc_effective'].fillna(df['is_h1n1_vacc_effective'].median())
```

```
df['is_h1n1_vacc_effective'].value_counts()
```

```
df['is_h1n1_risky'].unique()
```

```
sns.boxplot(df['is_h1n1_risky'])
```

```
df['is_h1n1_risky'] = df['is_h1n1_risky'].fillna(df['is_h1n1_risky'].median())
```

```
df.isnull().sum()
```

```
df['sick_from_h1n1_vacc'].unique()
```

```
sns.boxplot(df['sick_from_h1n1_vacc'])
```

```
df['sick_from_h1n1_vacc'] = df['sick_from_h1n1_vacc'].fillna(df['sick_from_h1n1_vacc'].median())
```

```
df['is_seas_vacc_effective'].unique()
```

```
df['is_seas_vacc_effective'] = df['is_seas_vacc_effective'].fillna(df['is_seas_vacc_effective'].median())
```

```
df['is_seas_risky'].unique()
```

```
df['is_seas_risky'] = df['is_seas_risky'].fillna(df['is_seas_risky'].median())
```

```
df['sick_from_seas_vacc'].unique()
```

```
df['sick_from_seas_vacc'] = df['sick_from_seas_vacc'].fillna(df['sick_from_seas_vacc'].median())
```

```
df['no_of_adults'].unique()
```

```
sns.boxplot(df['no_of_adults'])
```

```
df['no_of_adults'].idxmax()
```

```
df['no_of_adults'].fillna(df['no_of_adults'].median(),inplace=True)
```

```
df['no_of_children'].unique()
```

```
sns.boxplot(df['no_of_children'])
```

```
df['no_of_children'].fillna(df['no_of_children'].median(),inplace=True)
```

```
df.isnull().sum()
```

```
df.drop(['cont_child_undr_6_mnths'],axis=1,inplace=True)
```

```
df.dtypes
```

```
df.shape
```

```
df['age_bracket'].unique()
```

```
age_bracket_mapping = {
    '18 - 34 Years': 26,  # midpoint of 18-34
    '35 - 44 Years': 40,  # midpoint of 35-44
    '45 - 54 Years': 50,  # midpoint of 45-54
    '55 - 64 Years': 60,  # midpoint of 55-64
    '65+ Years': 70       # chosen representative value for 65+
}
```

```
df['age_bracket'] = df['age_bracket'].map(age_bracket_mapping)
```

```
df['age_bracket'].dtypes
```

```
df['sex'].unique()
```

```
df['sex'].value_counts()
```

```
df['sex'] =  df['sex'].map({'Female':2,'Male':1})
```

```
df.dtypes
```

Split data

```
x = df.drop(['h1n1_vaccine'],axis=1)
y = df['h1n1_vaccine']
```

```
x = df.drop(['h1n1_vaccine'],axis=1)
y = df['h1n1_vaccine']
```

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.3,random_state=120,stratify=y)
```

Model Selection

1. As data is binary-classification data and small we will implement Logistics Regression 2. As majority features are categorical we will implementing Decision Tree.
3. As data is classification and small data we will implementing Naive Bayes.
4. As SVC is excellent for binary classification and as our data is complex and has many features. We will implenting Support Vector Classification

Logistics Regression

```
from sklearn.linear_model import LogisticRegression
```

```
logi = LogisticRegression()
```

```
logi.fit(x_train,y_train)
```

```
y_pred = logi.predict(x_test)
```

```
from sklearn.metrics import classification_report,confusion_matrix
```

```
confusion_matrix(y_test,y_pred)
```

```
print(classification_report(y_test,y_pred))
```

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt = DecisionTreeClassifier()
```

```
dt.fit(x_train,y_train)
```

```
y_pred = dt.predict(x_test)
```

```
print(classification_report(y_test,y_pred))
```

Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
```

```
nb = GaussianNB()
```

```
nb.fit(x_train,y_train)
```

```
y_pred = nb.predict(x_test)
```

```
print(classification_report(y_test,y_pred))
```

Svm

```
from sklearn.svm import  SVC
```

```
sv = SVC(kernel='linear')
```

```
sv.fit(x_train,y_train)
```

```
y_pred = sv.predict(x_test)
```

```
print(classification_report(y_test,y_pred))
```

Random_State

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier()
```

```
rf.fit(x_train,y_train)
```

```
y_pred =  rf.predict(x_test)
```

```
print(classification_report(y_test,y_pred))
```

Summary

As our data is for a binary classification task and is relatively small, we performed extensive exploratory data analysis (EDA) and preprocessing to prepare the data for modeling. Here are the steps we took:

Exploratory Data Analysis (EDA)

As our data is for a binary classification task and is relatively small, we performed extensive exploratory data analysis (EDA) and preprocessing to prepare the data for modeling. Here are the steps we took:

Exploratory Data Analysis (EDA)

Data Visualization: Created visualizations to understand the distribution and relationships of the features.

Summary Statistics: Calculated key statistics to identify trends and anomalies in the data.

Correlation Analysis: Analyzed the correlation between features to detect multicollinearity.

Preprocessing

Handling Missing Values: Imputed missing values to ensure complete data

Categorical Encoding: Converted categorical variables into numerical format using techniques like one-hot encoding.

Splitting Data: Divided the data into training and testing sets to evaluate model performance.

Model Performance After preparing the data, we tested several models to see which one performs the best. Here are the accuracy results:

Logistic Regression: 83%
Decision Tree: 75%
Naive Bayes: 78%
Support Vector Machine (SVM): 83%
Random Forest: 83%
The models with the best accuracy are Logistic Regression, SVM, and Random Forest, each achieving an accuracy of 83%.